

KOMPOSOS-III Decomposition into Orion Capabilities

Status: Concept / Planning Created: 2025-04-03 Purpose: Decompose KOMPOSOS-III into atomic, subject-agnostic Orion capabilities

Orion is a plugin framework for Python applications. Think of it like a kernel for building composable systems:

Capabilities: Atomic, single-responsibility plugins that do one thing well (like "read files" or "compute embeddings")

Flows: YAML pipelines that chain capabilities together like LEGO instructions

Hooks: Event triggers that capabilities can listen for and respond to

Hot-reloading: Add/update capabilities while the system runs

Dependency injection: Capabilities declare what they need (@requires), Orion wires it together

Why this matters: Instead of monolithic applications, you build small, reusable pieces that compose into different solutions. Change one piece without breaking others.

The Problem We're Solving

KOMPOSOS-III is a brilliant categorical reasoning system for predicting protein-protein interactions. But it's:

Tightly coupled to biology (ESM-2 protein embeddings, gene names, amino acid sequences)

Monolithic (800+ line strategy file with 9 different prediction methods)

Hard to extend (adding a new strategy requires touching core code)

Single-domain (can't easily reuse the elegant categorical logic for code analysis or knowledge graphs)

The insight: The categorical reasoning (Kan extensions, Yoneda patterns, composition) is UNIVERSAL. It works on ANY graph structure. The biology stuff is just one instantiation.

Vision

Extract the categorical reasoning framework from KOMPOSOS-III into reusable Orion capabilities that can compose pipelines for any graph-structured domain: code analysis, knowledge graphs, biological systems, social networks, etc.

Before: One biology application After: A general-purpose categorical reasoning framework where biology is just one use case

Why This Matters

Original use case: Predict protein-protein interactions using category theory New use case: Analyze code composability in a self-coding agent using the same categorical reasoning

The core insight: Proteins, functions, concepts, and people are all just nodes in a graph with typed edges. The categorical inference strategies (Kan extensions, composition, etc.) are UNIVERSAL - they work on any graph structure.

Concrete example:

Biology: TP53 → activates → KRAS (protein interaction)

Code: parse_config → imports → json_decoder (function dependency)

Knowledge: Newton → influenced → Einstein (concept evolution)

The pattern $A \rightarrow B \rightarrow C$ implies $A \rightarrow C$ (composition) regardless of domain. The strategies are the same; only the data changes.

Value of decomposing into Orion:

Reuse: Write categorical strategies once, apply to any domain

Compose: Mix and match strategies like LEGO blocks via flows

Extend: Add new strategies without touching existing code

Hot-reload: Update analysis while system runs (critical for self-coding agents)

Test: Each capability tested independently

Share: Contribute strategies back to original KOMPOSOS project

The Decomposition Strategy

What Are "Strata"?

Strata are layers of abstraction. Lower strata provide foundational primitives; higher strata build complex functionality on top. This is the "kernel architecture" - like an operating system:

Stratum 6: Output Layer (scoring, filtering)

↑

Stratum 5: Coherence Layer (validation, aggregation)

↑

Stratum 4: Categorical Strategies (the 9 inference methods)

↑

Stratum 3: Graph Primitives (traversal, analytics)

↑

Stratum 2: Data/Storage (graph store, embeddings)

↑

Stratum 1: Core (Orion framework itself)

Why layer?

Lower strata don't know about higher strata (dependency direction: down only)

Swap out strata without breaking others (change embeddings, strategies still work)

Test each layer independently

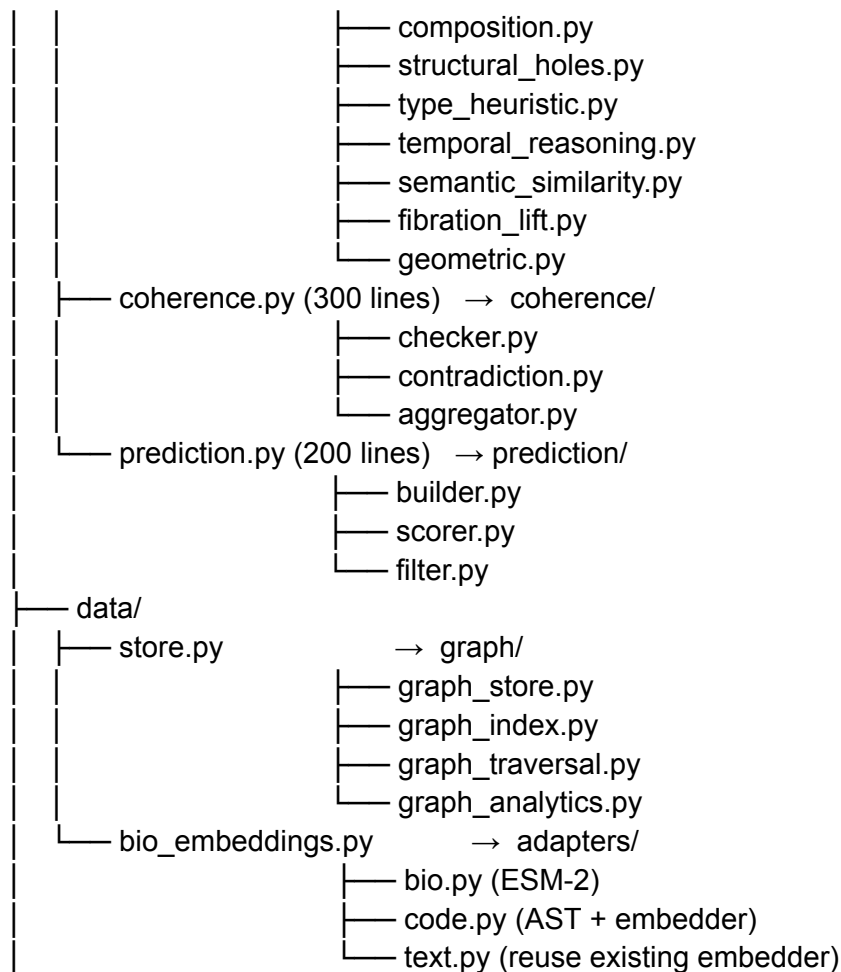
Reason about system in pieces

From Monolith to Capabilities

KOMPOSOS-III/

~/orion/capabilities/

```
|— oracle/
|   |— strategies.py (800 lines) → categorical/
|   |   |— kan_extension.py
|   |   |— yoneda_pattern.py
```



Capability Breakdown

What is a capability? A capability is a self-contained plugin that provides one well-defined piece of functionality. It:

Declares what it provides (`provides={"graph_store"}`)

Declares what it needs (`@requires("embeddings")`)

Exposes hooks for flows to call

Can be hot-reloaded without restarting the system

Why atomic? Small capabilities are easier to test, understand, and reuse. You can swap out one implementation without touching others.

Stratum 2: Data/Storage

Purpose: Store and query graph structures (nodes and edges). These are the foundational data structures that all higher-level strategies operate on.

graph_store.py

"""Universal graph storage for nodes and typed edges.

Methods:

```
add_node(id, type, metadata) -> None
add_edge(source, target, type, metadata) -> None
get_node(id) -> Node | None
get_edges(source?, target?, type?) -> list[Edge]
outgoing(node_id) -> list[Edge]
incoming(node_id) -> list[Edge]
"""
```

```
# graph_index.py
```

```
"""Fast adjacency indexing for graph operations.
```

```
Methods:
```

```
    build_outgoing() -> Dict[node_id, list[Edge]]
    build_incoming() -> Dict[node_id, list[Edge]]
    get_neighbors(node_id, direction) -> list[node_id]
"""
```

```
# similarity.py
```

```
"""Cosine similarity for embeddings (already exists in embedder).
```

```
Methods:
```

```
    similarity(vec_a, vec_b) -> float
"""
```

```
Stratum 3: Graph Primitives
```

```
# graph_traversal.py
```

```
"""Path finding and graph traversal.
```

```
Methods:
```

```
    bfs(start, max_depth) -> list[Path]
    dfs(start, max_depth) -> list[Path]
    find_paths(source, target, max_hops) -> list[Path]
    find_triangles() -> list[Triangle]
"""
```

```
# graph_analytics.py
```

```
"""Graph metrics and analysis.
```

```
Methods:
```

```
    degree Centrality(node_id) -> float
    clustering_coefficient(node_id) -> float
    find_hubs(threshold) -> list[node_id]
"""
```

```
Stratum 4: Categorical Strategies
```

What are categorical strategies? These are inference methods from category theory that predict "what edges should exist in this graph?" Each strategy uses a different mathematical principle to make predictions.

Why so many strategies? Different strategies catch different patterns. Like having multiple experts look at the same problem - each sees something the others miss. The coherence layer later combines their predictions and filters out contradictions.

Strategies are domain-agnostic: They work on ANY graph (proteins, code, concepts) because they operate on graph structure, not content. The "domain adapter" layer (embeddings) translates content into structure the strategies can reason about.

Each strategy is independent and domain-agnostic:

```
# kan_extension.py
"""Predict via Kan extension (colimit computation).
```

Logic: If similar objects connect to target, source should too.

```
@requires("graph_store", "similarity")
predict(source, target, graph) -> Prediction
"""
```

```
# yoneda_pattern.py
"""Predict via Yoneda lemma (Hom-set similarity).
```

Logic: Objects with same edge patterns are similar.

```
@requires("graph_store")
predict(source, target, graph) -> Prediction
"""
```

```
# composition.py
"""Predict via transitive closure.
```

Logic: $A \rightarrow B \rightarrow C$ implies $A \rightarrow C$.

```
@requires("graph_store", "graph_traversal")
predict(source, target, graph) -> Prediction
"""
```

```
# structural_holes.py
"""Find triangles that should close.
```

Logic: If $A \rightarrow C$ and $B \rightarrow C$ exist, $A \rightarrow B$ might exist.

```
@requires("graph_store", "graph_traversal")
predict(source, target, graph) -> Prediction
"""
```

```
# type_heuristic.py
"""Predict using type constraints.
```

Logic: Certain edge types connect certain node types.

```
@requires("graph_store")
predict(source, target, graph) -> Prediction
"""
```

```
# temporal_reasoning.py
"""Predict using temporal metadata.
```

Logic: Earlier things may influence later things.

```
@requires("graph_store")
predict(source, target, graph) -> Prediction
"""
```

```
# semantic_similarity.py
"""Predict using embedding similarity.
```

Logic: Similar things should have similar connections.

```
@requires("graph_store", "embeddings")
predict(source, target, graph) -> Prediction
"""
```

```
# fibration_lift.py
"""Predict via fibration structure.
```

Logic: Patterns preserve across fibers (type groups).

```
@requires("graph_store")
predict(source, target, graph) -> Prediction
"""
```

```
# geometric.py
"""Predict using curvature (Ricci).
```

Logic: Nodes in same geometric region likely connect.

```
@requires("graph_store", "graph_analytics")
predict(source, target, graph) -> Prediction
"""
```

Stratum 5: Coherence Layer

```
# coherence/checker.py
"""Sheaf-theoretic coherence validation.
```

Logic: Predictions must agree on overlaps.

```
@requires("embeddings")
check_coherence(predictions) -> CoherenceResult
"""
```

```
# coherence/contradiction.py
"""Detect semantic contradictions.
```

Logic: Antonym pairs, incompatible relations.

```
detect_contradiction(pred_a, pred_b) -> str | None
"""
```

```
# coherence/aggregator.py
"""Aggregate votes from multiple strategies.
```

Logic: Multiple confirmations boost confidence.

```
aggregate_predictions(predictions) -> PredictionBatch
"""
```

Stratum 6: Output Layer

```
# prediction/builder.py
"""Assemble prediction objects.
```

```
build(source, target, relation, confidence, evidence) -> Prediction
"""
```

```
# prediction/scorer.py
"""Rank predictions by confidence.
```

```
score(predictions) -> list[Prediction]
"""
```

```
# prediction/filter.py
"""Threshold and deduplicate.
```

```
filter(predictions, min_confidence, deduplicate) -> PredictionBatch
"""
```

Domain Adapters (Pluggable)

What are domain adapters? The categorical strategies work on graphs and similarity scores - they don't care WHERE the data comes from. Domain adapters translate raw domain data into the format strategies need.

The problem: How do you compare "how similar are two proteins?" vs "how similar are two functions?" You need different embedding models.

The solution: Domain adapters provide a standard interface:

```
embed(entity) -> vector      # Convert to numbers
similarity(a, b) -> float     # Compare two entities (0-1)
The strategy just calls:
```

```
sim = await self.embeddings.similarity(entity_a, entity_b)
It doesn't know or care if:
```

```
entity_a is a protein (ESM-2 embeddings)
entity_a is a function (CodeBERT embeddings)
entity_a is a concept (sentence-transformer embeddings)
Swapping domains: Change the adapter in the flow, keep the strategy.
```

```
# adapters/bio.py
"""Biology: ESM-2 protein embeddings.
```

```
Input: Gene symbol "TP53"
Output: 1280-dim protein structure embedding
"""
```

```
# adapters/code.py
"""Code: AST + docstring embeddings.
```

```
Input: Function name "parse_config"
Output: Hybrid structural + semantic embedding
"""
```

```
# adapters/text.py
"""Text: Reuse existing embedder capability.
```


Input: Concept "quantum_entanglement"

Output: Semantic embedding via embedder capability

""""

Pipeline Examples

What is a flow? A flow is a YAML file that defines a pipeline by chaining capabilities together.

Think of it like:

Declare what you need (requires) - Orion loads those capabilities

Define the steps (steps) - Each step calls a hook from a capability

Trigger it (trigger) - What event starts the flow

Why flows?

No code required: Define analysis pipelines as YAML

Reusable: Same flow works on different data

Composable: Mix capabilities like LEGO blocks

Debuggable: See exactly what steps run, in what order

Swappable: Change one capability without rewriting the pipeline

Example: Want to swap ESM-2 embeddings for CodeBERT? Change one line in the flow, not the strategy code.

Biology Pipeline (Original Use Case)

Goal: Given known protein interactions, predict new ones.

How it works:

Load protein data (genes, sequences, known interactions)

Build graph where nodes = proteins, edges = interactions

Run categorical strategies to predict missing edges

Check that predictions agree (coherence)

Return high-confidence predictions

~/.orion/flows/biology_interaction_prediction.yaml

name: biology_interaction_prediction

description: Predict protein-protein interactions

steps:

- name: load_proteins

 - capability: adapters/bio.py

- name: build_graph

 - capability: graph_store

 - hooks: [add_nodes, add_edges]

- name: index_graph

 - capability: graph_index

hooks: [build_outgoing, build_incoming]

- name: kan_analysis
capability: categorical/kan_extension
config:
 similarity_adapter: bio
- name: yoneda_analysis
capability: categorical/yoneda_pattern
- name: composition_analysis
capability: categorical/composition
- name: validate_coherence
capability: coherence/checker
- name: aggregate
capability: coherence/aggregator
- name: output
capability: prediction/filter
config:
 min_confidence: 0.6

Code Composability Pipeline (New Use Case)

Goal: Analyze a codebase to find missing abstractions, repeated patterns, and composability issues.

How it works:

Parse code into AST (abstract syntax tree) - extracts functions, imports, call relationships

Build graph where nodes = functions/modules, edges = calls/imports

Run categorical strategies:

Kan extension: "If 5 modules all use the same 3-step pattern, extract it as a shared capability"

Yoneda: "These modules have similar import patterns - should they share an interface?"

Composition: "A calls B calls C - should A call C directly?"

Structural holes: "A and B both depend on C but don't interact - should they?"

Validate predictions are coherent (not contradictory)

Return suggestions for improving composability

Why this matters for self-coding agents: The agent can analyze its own generated code and receive feedback like:

"You duplicated this pattern 3 times - extract a capability"

"These 5 functions are similar - unify their interfaces"

"This chain is too long - simplify the abstraction"

This creates a self-improvement loop where the agent learns to write more composable code.

```
# ~/.orion/flows/code_composability_analysis.yaml
```

```
name: code_composability_analysis
```

```
description: Analyze code for missing abstractions and composability issues
```

```
steps:
```

```
- name: parse_ast
```

```
  capability: ast_parser
```

```
  config:
```

```
    extract: [imports, calls, class_hierarchy]
```

```
- name: build_graph
```

```
  capability: graph_store
```

```
  hooks: [add_nodes, add_edges]
```

```
- name: index_graph
```

```
  capability: graph_index
```

```
- name: kan_analysis
```

```
  capability: categorical/kan_extension
```

```
  config:
```

```
    similarity_adapter: code
```

```
  purpose: Find repeated capability chains that should be extracted
```

```
- name: yoneda_analysis
```

```
  capability: categorical/yoneda_pattern
```

```
  purpose: Detect implicit interfaces across modules
```

```
- name: composition_analysis
```

```
  capability: categorical/composition
```

```
  purpose: Find  $A \rightarrow B \rightarrow C$  chains that could simplify to  $A \rightarrow C$ 
```

```
- name: structural_holes
```

```
  capability: categorical/structural_holes
```

```
  purpose: Find missing connections between related modules
```

```
- name: validate_coherence
```

```
  capability: coherence/checker
```

```
  purpose: Check for design pattern violations
```

```
- name: aggregate
```

```
  capability: coherence/aggregator
```

```
- name: output_feedback
  capability: prediction/filter
  config:
    min_confidence: 0.6
  emit:
    event: composability_feedback
```

Self-Coding Agent Integration

The capabilities can analyze the capability system itself:

```
@plugin
```

```
class CapabilityGraphAnalyzer(Plugin):
```

```
    """Analyze Orion capability dependency graph for composability."""
```

```
    @requires("graph_store")
```

```
    @requires("kan_extension")
```

```
    @requires("composition")
```

```
    async def analyze_myself(self):
```

```
        # Build graph of capability dependencies
```

```
        capabilities = self.core.list_capabilities()
```

```
        graph = await self.build_dependency_graph(capabilities)
```

```
        # Run categorical strategies
```

```
        missing = await self.kan_extension.predict(
```

```
            source="file_reader",
```

```
            target="http_client",
```

```
            graph=graph
```

```
        )
```

```
        # Suggest improvements
```

```
        if missing.confidence > 0.7:
```

```
            emit("composability_improvement", {
```

```
                "suggestion": f"Extract shared abstraction between {missing.source} and
```

```
{missing.target}",
```

```
                "evidence": missing.evidence
```

```
            })
```

Key Design Principles

Single Responsibility: Each capability does one thing well

Domain Agnostic: Core strategies work on any graph structure

Pluggable Adapters: Domain-specific embeddings are swappable

Hook-Based Composition: Flows chain capabilities via hooks

Event-Driven: Strategies emit events, coherence listens

Reusable: Same capabilities compose for biology, code, knowledge graphs

Existing Infrastructure to Leverage

Already in ~/.orion/capabilities/

✓ embedder # Text embeddings (reusable for text domain)

✓ inference_backend # Multiple backends (ollama, hf, litellm)

✓ tokeniser # Tokenization capabilities

Only need to build:

Categorical strategies (domain-agnostic)

Code embeddings adapter (AST + docstring)

Graph primitives (if not using external library)

References

Original project: <https://github.com/Jayhawk314/KOMPOSOS-III-ALPHA>

Key papers: See KOMPOSOS-III repository (TECHNICAL_REPORT.md, SUPPLEMENTARY_MATERIALS.md)

Orion capabilities: ~/.orion/capabilities/ (existing patterns)

Orion philosophy: docs/manifests/Framework_Philosophy.md

Next Steps

Proof of Concept: Implement kan_extension with graph_store

Validate Pattern: Test with simple graph

Expand: Add remaining categorical strategies

Domain Test: Build code embeddings adapter

Integrate: Connect with self-coding agent workflow

Summary: The Big Picture

We're taking a brilliant but domain-specific system (KOMPOSOS-III for protein interactions) and extracting the universal patterns into reusable building blocks (Orion capabilities).

What You Get

Before: One biology application

KOMPOSOS-III

├── Hardcoded biology data

├── Monolithic strategy file

└── Single domain use case

After: A general-purpose categorical reasoning framework

Orion Capabilities

├── 9 categorical strategies (domain-agnostic)

├── Pluggable domain adapters (bio, code, text, ...)

├── Composable flows (YAML pipelines)

└── Hot-reloadable architecture

Three Use Cases, Same Framework

Domain	Nodes	Edges	Goal	Flow
--------	-------	-------	------	------

Biology	Proteins	Interactions	Predict new interactions	
---------	----------	--------------	--------------------------	--

biology_interaction.yaml

Code	Functions	Calls/imports	Find missing abstractions	code_composability.yaml
------	-----------	---------------	---------------------------	-------------------------

Knowledge Concepts Influences Trace idea evolution concept_evolution.yaml
Same strategies, different data.

How to Contribute Back

The original KOMPOSOS-III project (<https://github.com/Jayhawk314/KOMPOSOS-III-ALPHA>) would benefit from this refactoring:

Core package (domain-agnostic) - The categorical reasoning strategies

Biology package (existing code) - Uses core